

JSON & Assertions Reference

Read JSON, navigate it, and assert on it with confidence.

Most modern APIs speak JSON, so reading it and asserting on it is the core skill of API testing. This reference covers JSON's data types, how to navigate a payload with dot and bracket paths, the assertions you will use most, and a note on validating an entire response against a schema.

JSON Data Types

Type	Example	Notes
String	"Ada Lovelace"	Always double quotes, never single
Number	42 or 3.14	No quotes; integers and floats, no distinction
Boolean	true / false	Lowercase, unquoted
Null	null	Explicit absence of a value
Object	{ "id": 1, "name": "Ada" }	Unordered set of key/value pairs
Array	[1, 2, 3]	Ordered list; can hold any types

Sample Payload

The examples below navigate this response body: { "id": 1, "name": "Ada", "active": true, "roles": ["admin", "tester"], "address": { "city": "London", "postcode": "EC1A" } }

- id — a number (1)
- name — a string ("Ada")
- active — a boolean (true)
- roles — an array of strings (["admin", "tester"])
- address — a nested object with city and postcode

Path Navigation

Use dot notation for object keys and bracket notation for array indices (zero-based) or awkward keys.

Path	Resolves to	Note
json.id	1	Dot notation for a top-level key
json.address.city	"London"	Chain dots to go deeper
json.roles[0]	"admin"	Bracket + index for arrays (starts at 0)
json.roles.length	2	Array length
json.roles[json.roles.length - 1]	"tester"	Last element
json['address']['postcode']	"EC1A"	Bracket notation for keys works too

Common Assertions

Shown in Postman/Chai style; the same ideas apply in any test framework.

Check	Assertion
Status is 200	<code>pm.response.to.have.status(200);</code>
Field equals a value	<code>pm.expect(json.name).to.eql('Ada');</code>
Field exists	<code>pm.expect(json).to.have.property('id');</code>
Type is correct	<code>pm.expect(json.id).to.be.a('number');</code>
Boolean is true	<code>pm.expect(json.active).to.be.true;</code>
Array length	<code>pm.expect(json.roles).to.have.lengthOf(2);</code>
Array contains value	<code>pm.expect(json.roles).to.include('admin');</code>
Nested value	<code>pm.expect(json.address.city).to.eql('London');</code>

JSON Schema Validation

Field-by-field assertions confirm specific values; schema validation confirms the whole shape at once — which keys exist, their types, and which are required. It is the fastest way to catch a contract change (a renamed or dropped field) that individual assertions might miss.

- Define a schema describing the expected types and required fields.
- In Postman: `pm.test('schema is valid', () => { pm.expect(pm.response.json()).to.be.jsonSchema(schema); });`
- Great for regression: one check guards the entire response contract.

INSIDE STLC PRO TIP

Combine both approaches. Use schema validation as a broad contract guard on the whole payload, then add a handful of targeted value assertions for the fields that carry business meaning. Together they catch both structural breaks and wrong data.